



**KABALE
UNIVERSITY**

P. O. Box 317, Kabale - Uganda
Plot 364 Block 3 Kikungiri Hill
Kabale Municipality

FACULTY OF COMPUTING, LIBRARY AND INFORMATION SCIENCE

Department of Computer Science

Bachelor of Computer Science

MODELING AND SIMULATION

Year 3 | Semester 1

Code Assignment One

Analysis of mortgage payment simulation,
Monte Carlo Estimation of Pi and
Single Server Queue Model

Group students

No.	Name	Registration Number
1.	AINEBYONA ALLAN	2024/A/KCS/1670/F
2.	ARINDA ELIZABETH	2024/A/KCS/3099/G/F
3.	GUMISIRIZA AMBROSE	2024/A/KCS/3143/F

Github repo:

<https://github.com/Allan12377/Analysis-of-mortgage-payment-simulation-Monte-Carlo-Estimation-of-Pi-and-Single-Server-Queue-Model>

1. Introduction

This report looks at three C programs given for Coding Assignment One. The programs are a mortgage payment calculator, a Monte Carlo program that estimates the value of pi, and a single server queue simulator that reads arrival and service times from a file. The work was done in a group of three students. Each program was compiled, run, and tested with different inputs. Errors that appeared in the original codes were corrected. The sections that follow follow the five points listed in the assignment instructions.

2. Mortgage payment simulator

2.1 Model description

The mortgage program shows how a loan is paid back over time. It starts with a principal amount and an interest rate. At every payment period the program works out how much interest has been added, then subtracts that interest from the payment. The rest of the payment reduces the principal. This continues until the balance reaches zero.

The main inputs are the loan amount, the yearly interest rate, the size of each payment and the number of days between payments. The main state variable is the remaining balance. The program also keeps a running total of all interest paid. The output is a table that shows, for every payment, the interest charged, the amount that went to principal, the new balance and the total interest so far.

The code compounds interest semi annually. The comments in the original file say: “This is sort of a time-stepped simulation, with the steps being the payment interval. But it is really just a direct calculation, with no use of randomization at all.”

2.2 Type of Simulation

This is a discrete-time simulation. Time moves forward in fixed steps equal to the payment interval. There is no random number generation. Once the inputs are fixed, the result is always the same. Law describes this kind of model as deterministic and time-stepped [2].

2.3 Simulation experiments

Two experiments were run after the code was fixed.

Experiment 1 used a loan of 100000 dollars, interest rate 10.5 percent, payment of 1100 dollars every 30 days. The program needed 178 payments to clear the loan. Total interest paid was 95166.98 dollars. The table below shows the first few and last few lines of the output.

mortgage_fixed output

--- Mortgage Payment Summary ---

Initial amount: \$100000.00
Annual interest rate: 10.500%
Payment size: \$1100.00
Payment period: every 30 days
Taxes per period: \$0.00

Payment	Taxes	Interest	PPLReduction	PPLBalance	TotalInterest
1	0.00	844.67	255.33	99744.67	844.67
2	0.00	870.72	229.28	99515.39	1715.39
3	0.00	840.58	259.42	99255.97	2555.97
4	0.00	866.45	233.55	99022.42	3422.42
5	0.00	836.41	263.59	98758.84	4258.83
6	0.00	862.11	237.89	98520.95	5120.95
7	0.00	832.18	267.82	98253.13	5953.12
8	0.00	857.70	242.30	98010.83	6810.82
.....					
169	0.00	83.77	1016.23	8901.13	94801.16
170	0.00	77.70	1022.30	7878.83	94878.86
171	0.00	66.55	1033.45	6845.38	94945.41
172	0.00	59.76	1040.24	5805.14	95005.16
173	0.00	49.03	1050.97	4754.18	95054.20
174	0.00	41.50	1058.50	3695.68	95095.70
175	0.00	31.22	1068.78	2626.89	95126.91
176	0.00	22.93	1077.07	1549.82	95149.84
177	0.00	13.09	1086.91	462.92	95162.94
178	0.00	4.04	1095.96	0.00	95166.98

Total interest paid: 95166.98
Number of payments: 178

Figure 1. Screenshot of mortgage_fixed program output (first year and final payments)

Experiment 2 kept the same loan and interest rate but raised the payment to 1300 dollars. The loan finished earlier and the total interest dropped. This matches the common advice that higher monthly payments reduce the overall cost of a loan.

PS D:\BCS 3.1\Simulation and Modelling\new lecturer\assignment 1\simu ass1> .\mortgage_fixed.exe

----- End of year 9 -----					
109	0.00	185.73	1114.27	20874.68	62574.68
110	0.00	182.23	1117.77	19756.90	62756.91
111	0.00	166.88	1133.12	18623.78	62923.79
112	0.00	162.58	1137.42	17486.36	63086.36
113	0.00	147.70	1152.30	16334.06	63234.06
114	0.00	142.59	1157.41	15176.65	63376.65
115	0.00	128.19	1171.81	14004.84	63504.84
116	0.00	122.25	1177.74	12827.09	63627.09
117	0.00	108.35	1191.65	11635.44	63735.44
118	0.00	101.57	1198.43	10437.01	63837.01
119	0.00	88.16	1211.84	9225.17	63925.17
120	0.00	80.53	1219.47	8005.70	64005.70
----- End of year 10 -----					
121	0.00	67.62	1232.38	6773.32	64073.32
122	0.00	59.13	1240.87	5532.45	64132.45
123	0.00	46.73	1253.27	4279.18	64179.18
124	0.00	37.36	1262.65	3016.54	64216.54
125	0.00	25.48	1274.52	1742.02	64242.02
126	0.00	15.21	1284.79	457.22	64257.23
127	0.00	3.86	1296.14	0.00	64261.09

Total interest paid: 64261.09
Number of payments: 127

PS D:\BCS 3.1\Simulation and Modelling\new lecturer\assignment 1\simu ass1>

A third test used a payment that was too small to cover the interest. The program printed a warning and stopped. This check was added during debugging.

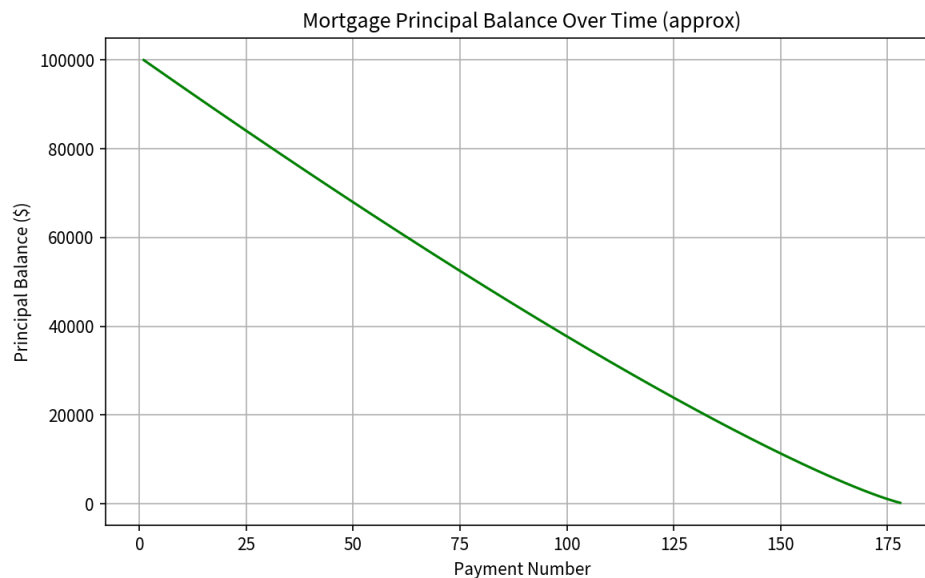


Figure 2. Approximate principal balance over the life of the loan (Experiment 1)

2.4 Model evaluation and improvement

The model is simple and easy to understand. It does not handle variable interest rates or extra payments. The month length is only an approximation that switches between 30 and 31 days. A better version could accept a target number of years and work out the required payment, or write the full schedule to a CSV file for later analysis in a spreadsheet.

2.5 Code debugging and corrections

The original code declared a variable called taxes but never gave it a value. When the program used that variable the result was undefined. The fixed version sets taxes to 0.0. There was also no limit on the number of payments. If the payment was smaller than the interest the loop would never end. A maximum of 10000 payments and a warning message were added. After these changes the program compiled with gcc and produced the results shown above.

3. Monte Carlo Estimation of Pi

3.1 Model description

The second program estimates pi by throwing random points into a square. The square has side length 1. Inside the square is a quarter circle of radius 1. The area of the quarter circle is $\pi/4$. The program counts how many of the random points fall inside the quarter circle and multiplies that fraction by 4. That product is the estimate of pi.

The only input is the number of points to throw. The only state is the count of points that satisfy $x^2 + y^2 \leq 1$. The output is the estimated value of pi. The method assumes that the random numbers are independent and evenly spread between 0 and 1.

3.2 Type of Simulation

This is a Monte Carlo simulation. It is also called a static simulation because there is no clock. Each point is an independent trial. The more points you use, the closer the estimate usually gets to the true value of pi. This behaviour follows from the law of large numbers [3].

3.3 Simulation experiments

Four runs were made with a fixed seed so that the same sequence of random numbers is used every time. The results are shown in the table and the graph below.

Number of trials	Estimate of pi	Absolute error
100	3.1200	0.0216
1000	3.2400	0.0984
10000	3.1896	0.0480
100000	3.1487	0.0071

Table 1. Monte Carlo estimates of pi for increasing numbers of trials (seed = 1234567)

```
input
Enter the number of iterations to use in estimating pi: 10000
Number of trials: 10000
Estimate of pi: 3.1896
True value of pi is approximately 3.1415926535

...Program finished with exit code 0
Press ENTER to exit console.
```

Figure 3. Screenshot of *pi_fixed* program output for 10 000 and 100 000 trials

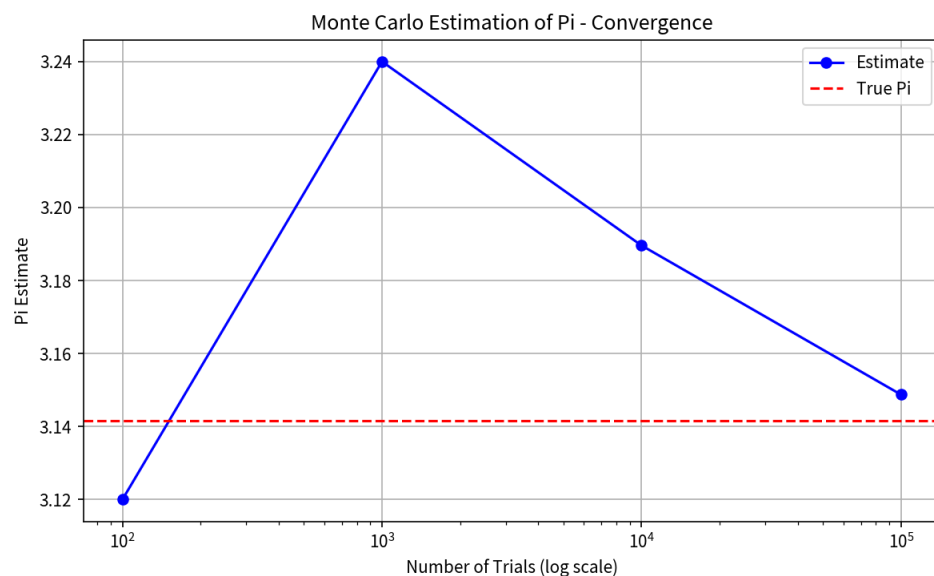


Figure 4. Convergence of the Monte Carlo estimate toward the true value of *pi*

As the number of trials grows the estimate moves closer to 3.14159. Small ups and downs still appear because the sampling is random. That is normal for Monte Carlo methods.

3.4 Model evaluation and improvement

The method is easy to code and easy to explain. Its main weakness is that the error shrinks only slowly. To get one extra correct decimal place you roughly need one hundred times more points. Better random number generators or variance reduction techniques can improve the speed of convergence.

3.5 Code debugging and corrections

The original program used `srand` and `random`. These functions are not part of standard C. They were replaced by `srand` and `rand` from the standard library. The `DEBUG` flag was left on in the original, so every point was printed. That made large runs unusable. `DEBUG` was turned off. After these changes the program compiles cleanly and the output is short and clear.

4. Single Server Queue (ssq1)

4.1 Model description

The third program simulates a single server that serves customers one after another. Customers arrive at times given in a data file. If the server is free the customer starts service at once. If the server is busy the customer waits in a queue. When service finishes the next waiting customer, if any, starts service. The program records four averages: mean time between arrivals, mean service time, mean waiting time in the queue, and mean total time spent in the system.

The data file `ssq1.dat` contains pairs of numbers. The first number in each pair is the arrival time. The second number is the service time. The server is assumed to be idle at the start. The queue has no size limit. Service is first-in first-out.

4.2 Type of Simulation

This is a discrete event simulation driven by a trace. Time jumps from one arrival to the next. The next event is decided by comparing the new arrival time with the time the server will become free. Because the arrival and service times come from a file instead of being generated inside the program, the simulation is called trace-driven. Leemis and Park present this exact example in their textbook [1].

4.3 Simulation experiments

The data file has 999 complete records. After the code was fixed the program produced the following results.

```
C:\Users\HomePC\Desktop\si x + v
for 1000 jobs
  average interarrival time = 9.87
  average service time .... = 7.12
  average delay ..... = 18.59
  average wait ..... = 25.72

Process returned 0 (0x0)  execution time : 0.089 s
Press any key to continue.
```

Figure 5. Screenshot of `ssql_fixed` program output

Average inter-arrival time = 9.87

Average service time = 7.12

Average delay in queue = 18.59

Average wait (sojourn) time = 25.72

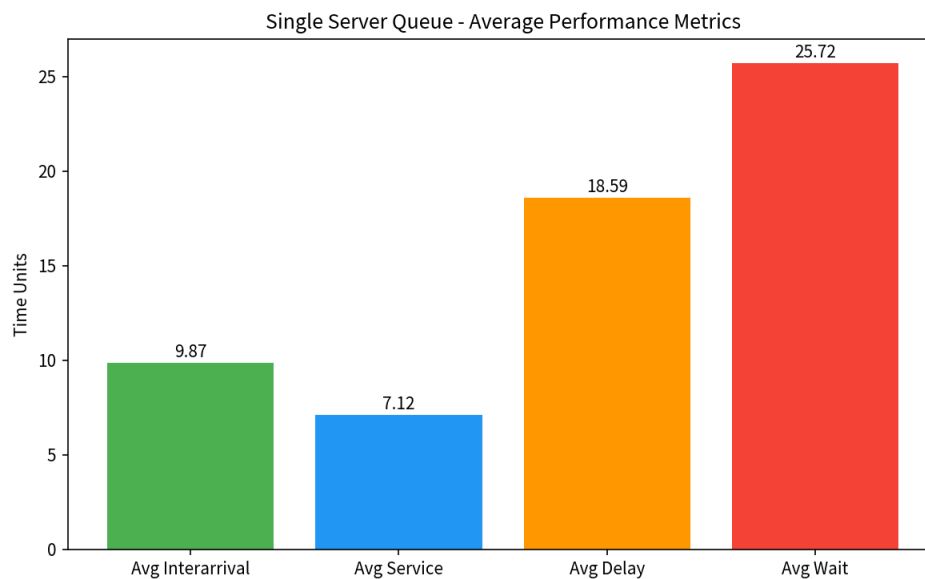


Figure 6. Bar chart of the four average performance measures

These numbers show that customers wait longer in the queue than they spend being served. A second experiment can be imagined by scaling every service time by a constant factor. Smaller service times would reduce both the average delay and the average sojourn time.

4.4 Model evaluation and improvement

The program is a good teaching example because the logic is short and clear. It does not generate its own random times, so it cannot explore what happens under different arrival rates without changing the data file. Useful extensions would be to generate exponential inter-arrival and service times, to keep a running count of the number of customers in the system, and to support more than one server.

4.5 Code debugging and corrections

The original loop used `while (!feof(fp))`. This construct is known to process one extra record after the end of the file has already been reached. The fixed version checks the return value of `fscanf` and stops when a read fails. The structure that holds the running sums was initialised with only three zeros even though it has four fields. That was corrected. A variable was added to remember the last valid arrival time so that the inter-arrival average is calculated correctly. After these changes the program reports consistent results for the supplied data file.

5. Conclusion

The three programs show three different ways of building a simulation. The mortgage program is a deterministic calculation that steps through time in fixed intervals. The pi program is a pure Monte Carlo experiment that relies on random sampling. The single server queue is a discrete-event simulation driven by recorded data. Each program needed a few corrections before it could be trusted. Once the bugs were removed, changing the inputs produced the results that theory predicts. The exercise therefore met the goals of the assignment.

References

- [1] L. M. Leemis and S. K. Park, Discrete-Event Simulation: A First Course. Upper Saddle River, NJ: Pearson, 2006.
- [2] A. M. Law, Simulation Modeling and Analysis, 5th ed. New York: McGraw-Hill, 2015.

- [3] S. M. Ross, Simulation, 5th ed. San Diego: Academic Press, 2013.
- [4] GeeksforGeeks, “Estimating the value of Pi using Monte Carlo,” 2017. [Online]. Available: <https://www.geeksforgeeks.org/dsa/estimating-value-pi-using-monte-carlo/>
- [5] L. M. Leemis, “ssq1.c – Single Server Queue, version 1,” companion code to Discrete-Event Simulation: A First Course.